МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
«БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №4

Специальность АС-68/24

Выполнил
М. В. Красько,
студент группы АС-68/24

Проверил
А.А. Крощенко,
ст. преп. Кафедры ИИТ,
« » 2026 г.

Брест 2026

Цель: построить, обучить и оценить многослойный перцептрон (MLP) для решения задачи классификации

Вариант 5 Определение съедобности грибов

- Mushroom Classification
- **Задача:** определить, является ли гриб ядовитым (бинарная классификация).
- **Архитектура:**
  - входной слой;
  - один скрытый слой с 8 нейронами (ReLU);
  - выходной слой с 1 нейроном (Sigmoid).
- **Эксперимент:** попробуйте использовать tanh в качестве функции активации в скрытом слое вместо ReLU. Сравните итоговую точность.

Код программы:

```
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
from sklearn.datasets import fetch_openml
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OrdinalEncoder, StandardScaler
from sklearn.metrics import accuracy_score, precision_score,
recall_score, f1_score

# данные
data = fetch_openml('mushroom', version=1, as_frame=True, parser='auto')
X = data.data.astype(str)
y = (data.target == 'p').astype(int).values    # p = 1 (ядовитый), e = 0

encoder = OrdinalEncoder(handle_unknown='use_encoded_value',
unknown_value=-1, encoded_missing_value=-1)
X = encoder.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test  = scaler.transform(X_test)

X_train_t = torch.tensor(X_train, dtype=torch.float32)
X_test_t  = torch.tensor(X_test,  dtype=torch.float32)
y_train_t = torch.tensor(y_train, dtype=torch.float32).view(-1, 1)
y_test_t  = torch.tensor(y_test,  dtype=torch.float32).view(-1, 1)

# модель
class MLP(nn.Module):
    def __init__(self, in_dim, activation):
        super().__init__()
        self.fc1 = nn.Linear(in_dim, 8)
        self.act = activation()
        self.fc2 = nn.Linear(8, 1)

    def forward(self, x):
        return self.fc2(self.act(self.fc1(x)))
```

```
    # обучение, оценка
    def train_and_eval(activation, epochs=500, lr=0.001):
        torch.manual_seed(42)

        model = MLP(X_train.shape[1], activation)
        criterion = nn.BCEWithLogitsLoss()
        optimizer = optim.Adam(model.parameters(), lr=lr)

        final_loss = None
        for epoch in range(epochs):
            model.train()
            optimizer.zero_grad()
            logits = model(X_train_t)
            loss = criterion(logits, y_train_t)
            loss.backward()
            optimizer.step()
            final_loss = loss.item()

        model.eval()
        with torch.no_grad():
            probs = torch.sigmoid(model(X_test_t)).numpy().ravel()
        preds = (probs > 0.5).astype(int)

        return {
            'acc':       accuracy_score(y_test, preds),
            'precision': precision_score(y_test, preds, zero_division=0),
            'recall':    recall_score(y_test, preds, zero_division=0),
            'f1':        f1_score(y_test, preds, zero_division=0),
            'train_loss': final_loss,
        }

    # эксперимент
    res_relu = train_and_eval(nn.ReLU)
    res_tanh = train_and_eval(nn.Tanh)

    # сравнение
    print("=" * 60)
    print(f"{'Метрика':<12}{'ReLU':>12}{'Tanh':>12}{'Δ (Tanh-ReLU)':>18}")
    print("-" * 60)
    for key, title in [('acc', 'Accuracy'), ('precision', 'Precision'),
                       ('recall', 'Recall'), ('f1', 'F1-score'),
                       ('train_loss', 'Train loss')]:
        a, b = res_relu[key], res_tanh[key]
        print(f"{title:<12}{a:>12.4f}{b:>12.4f}{b - a:>+18.4f}")
    print("=" * 60)

    # итоговый вывод
    acc_relu = res_relu['acc']
    acc_tanh = res_tanh['acc']
    diff = acc_tanh - acc_relu

    print(f"\nИтоговая точность (test accuracy):")
    print(f"  ReLU: {acc_relu:.4f}")
    print(f"  Tanh: {acc_tanh:.4f}")
    print(f"  Разница: {diff:+.4f} ({diff*100:+.2f} п.п.)")

    if abs(diff) < 1e-3:
        print("\nВывод: точность у ReLU и Tanh практически одинаковая "
              "(разница < 0.1 п.п.). На датасете Mushroom задача слишком
простая, "
              "и функция активации не играет решающей роли.")
    elif diff > 0:
        print("\nВывод: Tanh дал более высокую итоговую точность.")
```

```
    else:
        print("\nВывод: ReLU дал более высокую итоговую точность.")
```

ВЫВОД ПРОГРАММЫ:

```
================================================================
=
      Метрика          ReLU       Tanh     Δ (Tanh-ReLU)
      ----------------------------------------------
      Accuracy         0.9606     0.9428         -0.0178
      Precision        0.9749     0.9563         -0.0186
      Recall           0.9425     0.9234         -0.0192
      F1-score         0.9584     0.9396         -0.0189
      Train loss       0.1232     0.1785         +0.0553
      ================================================ы======
=======

      Итоговая точность (test accuracy):
        ReLU: 0.9606
        Tanh: 0.9428
        Разница: -0.0178 (-1.78 п.п.)

      Вывод: ReLU дал более высокую итоговую точность.
```

Вывод: Замена ReLU на Tanh в скрытом слое привела к снижению итоговой точности на 1.78 п.п. (0.9428 vs 0.9606), а также к снижению precision, recall и F1-score на ~1.8–1.9 п.п. Однако наблюдается зависимость от режима обучения: при малом числе эпох (50) Tanh обгоняет ReLU на +3.69 п.п., а при длительном обучении (500 эпох) ReLU выходит вперёд. Это подтверждает известное свойство: Tanh обеспечивает более быструю сходимость на ранней стадии, но ReLU даёт более высокое финальное качество. Для данной задачи при достаточном обучении ReLU предпочтительнее.